

Nick Ruider

Chair of Software Technology

Graphical Management of Multiple Runtime Models to Improve the Comprehension of Contextual Behavioral Programs

Defense of Master Thesis

Andreas-Pfitzmann-Bau // 25th June 2026

Structure

Defense of Master Thesis

Research Questions

Background

Behavioral Programming (*BP*)

Universal Variability Language (*UVL*)

Graphical Language Server Platform (*GLSP*)

State Of The Art

Existing Tools

Research Gap

Implementation

Extensions to FMBP & UVL Metamodel

UVL- & UVL-BP GLSP

Demo

Evaluation & Future Work

Research Questions

Motivation of this thesis

Graphical tool improving the comprehension of behavioral programs for multiple runtime models

1. How feasible is the **graphical visualization of UVL feature models** for improving the comprehension of contextual BP?
2. How can **BP concepts** be **integrated** into a graphical tool while maintaining **extensibility** for future features and analyses?
- 3.1. How can **changes to the feature model** during the execution of a behavioral program be automatically **reflected in the graphical representation**?
- 3.2. How can **user interactions** with the graphical model be **translated** into valid runtime modifications of the **underlying behavioral program**?

Background

Core domains of this thesis

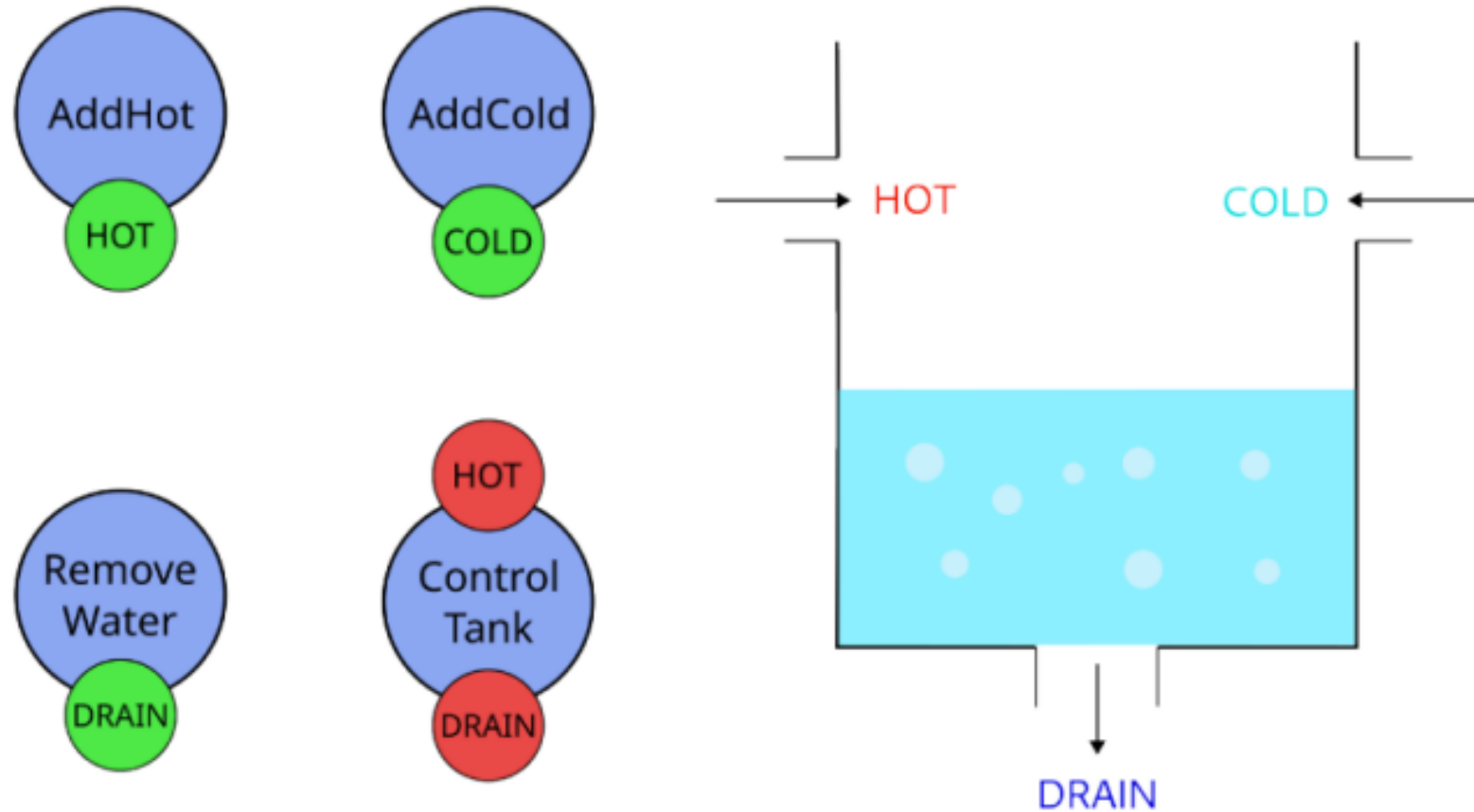
Behavioral Programming

Software Development Paradigm

- use cases and scenarios elevated to executable constructs [1]
 - behavioral threads (**b-threads**) as main building blocks
 - b-threads communication via a shared event mechanism at synchronization points
 - **b-event** types: *requested*, *waited-for* and *blocked*
- context-oriented extension handling dynamic environment changes [2]

Behavioral Programming

Water Tank Example



Water Tank Example [3]

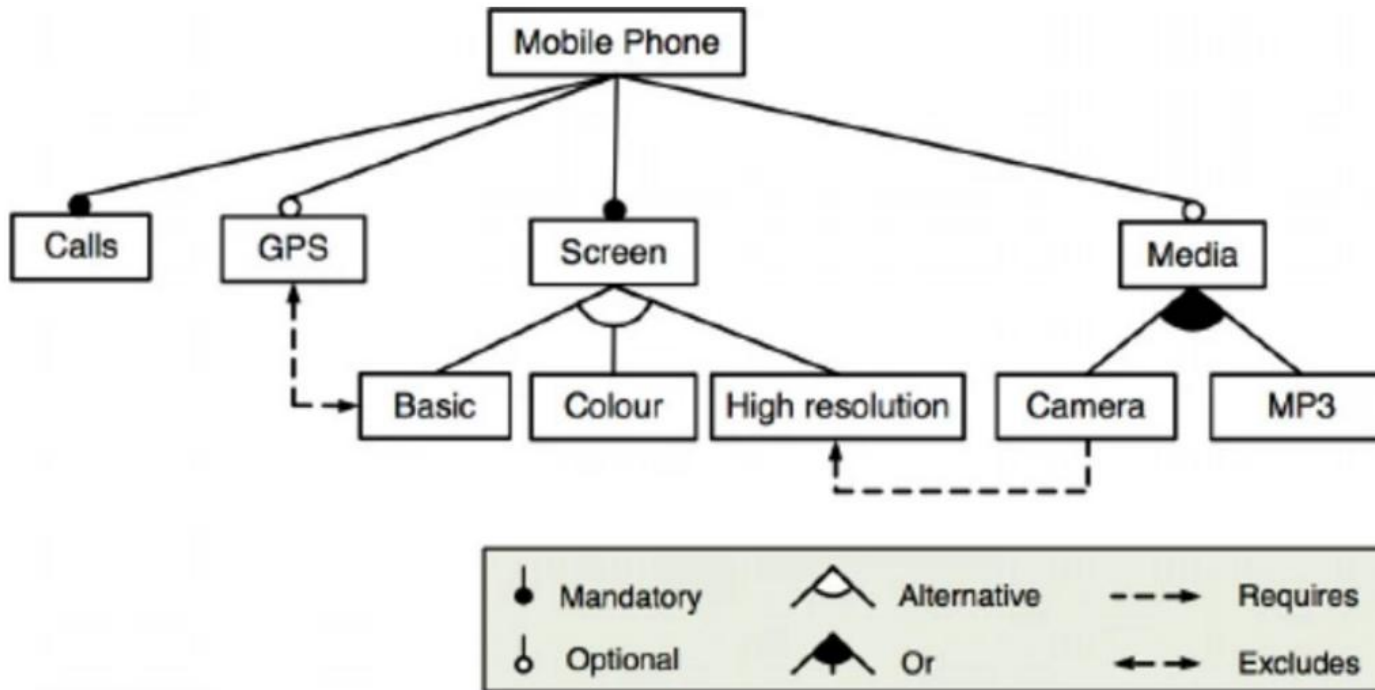
Universal Variability Language

Feature Modeling

- describing commonalities & variabilities with a software product family (Software Product Lines)
- commonly modeled as **feature trees** [4]
 - basic parent-child-relationships: *mandatory, optional, or & alternative*
 - cross-tree constraints, e.g.: *requires & excludes*
 - cardinality-based modeling: *feature cardinality & group cardinality*
 - extended feature models via *feature attributes*
- **Universal Variability Language** (UVL) as standardized textual exchange format [5]

Universal Variability Language

Mobile Phone Example



Feature tree for mobile phone example [4]

```

1 features
2   "Mobile Phone"
3   mandatory
4     Calls
5     Screen
6     alternative
7       Basic
8       Colour
9       "High resolution"
10  optional
11    GPS
12    Media
13    or
14      Camera
15      MP3
16 constraints
17   Camera => "High resolution" // requires
18   !(GPS & Basic) // excludes
  
```

UVL for mobile phone example

Graphical Language Server Platform

Technical foundation

- open-source **framework** hosted by the Eclipse Foundation
 - replicates decoupled architecture of Language Server Platform for **graphical modeling tools** [6]
- action-based communication via JSON-RPC protocol

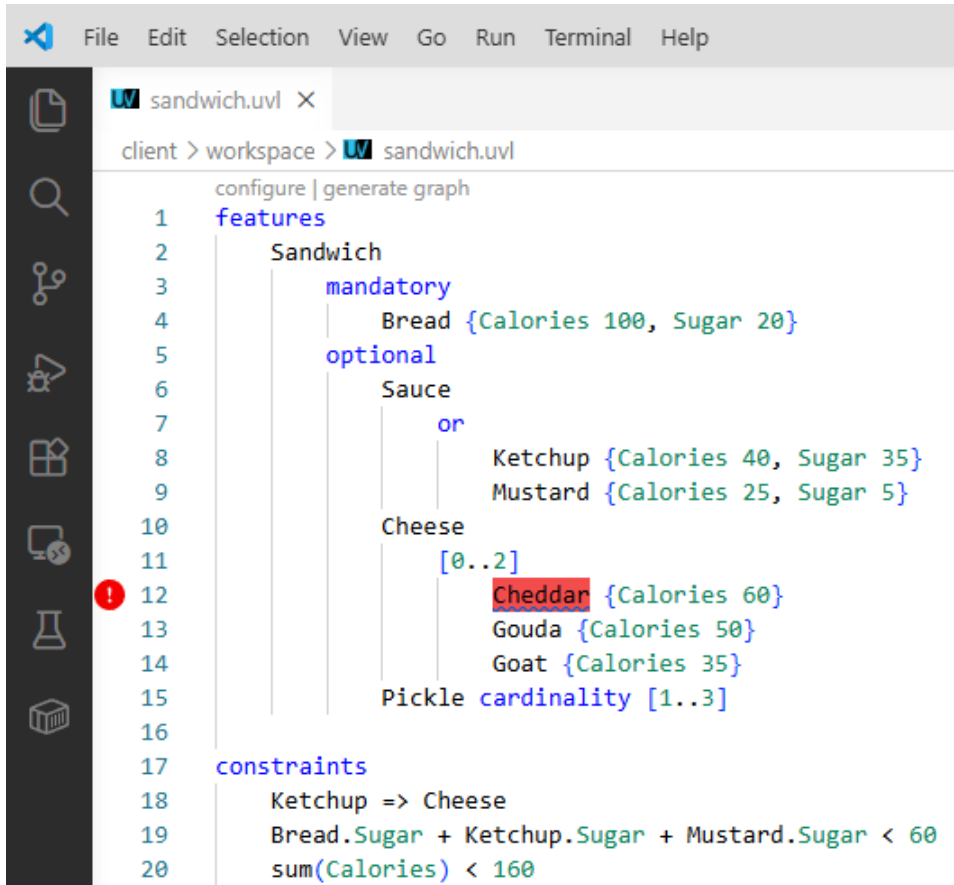


GLSP framework [6]

State Of The Art

Existing Tools & Research Gap

State Of The Art Feature Modeling

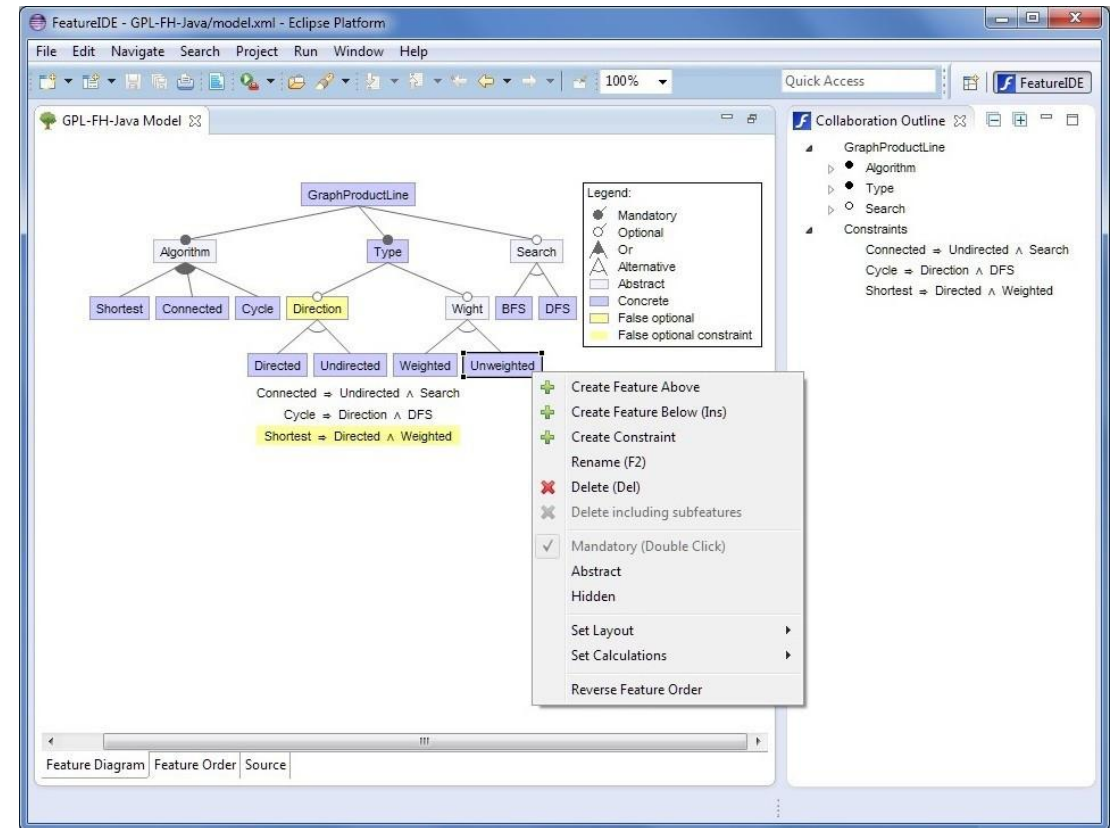


```

client > workspace > sandwich.uvl

configure | generate graph
features
1  Sandwich
2    mandatory
3    Bread {Calories 100, Sugar 20}
4    optional
5    Sauce
6      or
7      Ketchup {Calories 40, Sugar 35}
8      Mustard {Calories 25, Sugar 5}
9    Cheese
10     [0..2]
11     Cheddar {Calories 60}
12     Gouda {Calories 50}
13     Goat {Calories 35}
14   Pickle cardinality [1..3]
15
16 constraints
17 Ketchup => Cheese
18 Bread.Sugar + Ketchup.Sugar + Mustard.Sugar < 60
19 sum(Calories) < 160
20
  
```

VS Code with UVLS extension

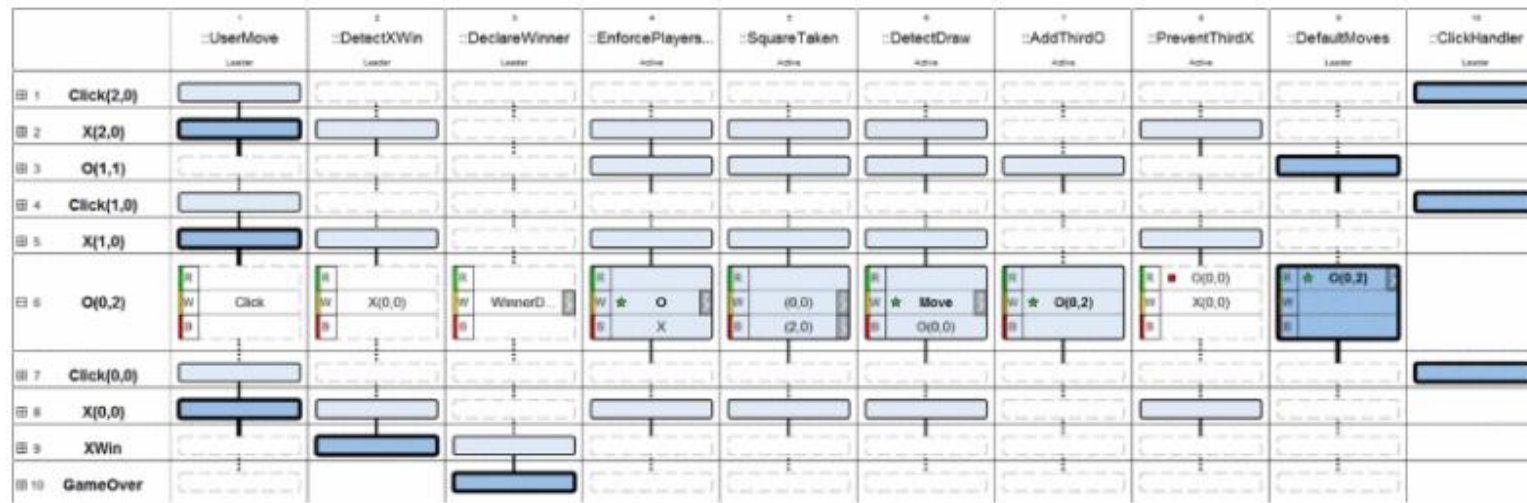


UI of FeatureIDE [7]

State Of The Art

Behavioral Programming

- Trace Visualization [8]
 - post-execution analysis tool
- BP Runtime Debugger [9]
 - hooks directly into behavioral program, including state information and runtime controls



TraceVis of Tic-tac-toe game example [8]

State Of The Art

Behavioral Programming

- **FMBP** as framework for comprehensibility at the architectural design phase [10]
- extends UVL to represent BP as an Architecture Description Language (ADL) [3]
 - b-threads → feature
 - b-events → attributes
 - context → additional top-level features *Config & Env*
 - additional aggregate constraint functions
- FMBP includes extended *UVLS* for BP & Python framework based on *BPpy* [10]
 - execute behavioral programs based on UVL model files

State Of The Art

Behavioral Programming

```
1  ∨ features
2  ∨      Watertank
3  ∨          [1..4]
4              AddHot {type 'BThread', HOT {type 'BEvent', requested true, priority 1}}
5              AddCold {type 'BThread', COLD {type 'BEvent', requested true, priority 1}}
6              RemoveWater {type 'BThread', DRAIN {type 'BEvent', requested true, priority 2}}
7              Finished {type 'BThread', FINISHED {type 'BEvent', requested true, priority 1}}
8          Env {type 'Env', temp 0.0, level 0.0}
9          Config {type 'Config', target_temp 20, target_level 50}
10
11 ∨ constraints
12     conflicting(HOT, COLD, DRAIN, FINISHED)
13     (Env.temp < Config.target_temp) => requested(HOT)
14     (Env.temp > Config.target_temp) => requested(COLD)
15     (Env.level > Config.target_level) <=> selected(DRAIN)
16     (Env.temp == Config.target_temp & Env.level == Config.target_level) <=> selected(FINISHED)
```

State Of The Art

Research Gap

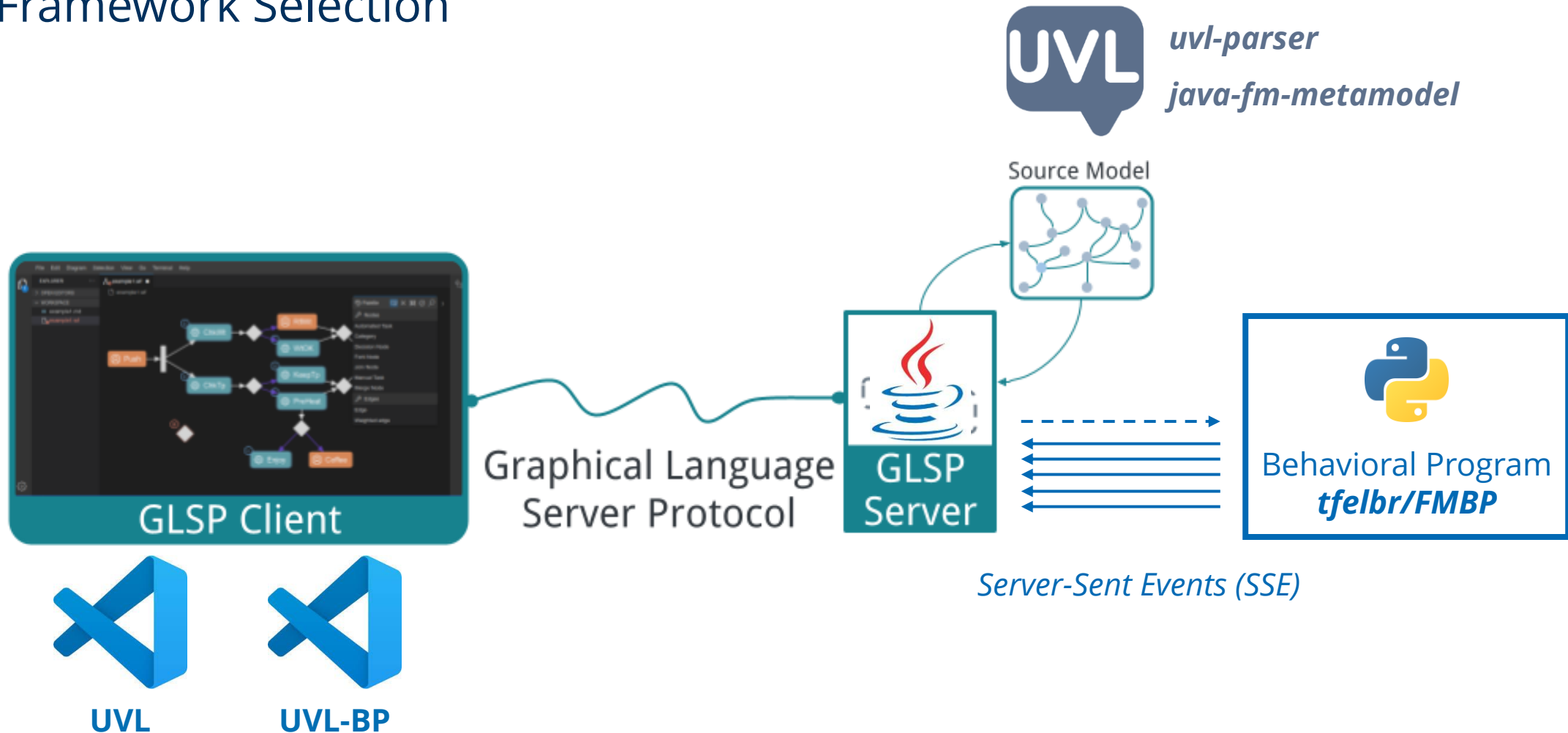
- FMBP as foundational work
 - presents ADL for behavioral programs
 - only textual representation
- existing graphical solutions of UVL not extendable on language-level
- GLSP as available framework for graphical modeling tools
 - extensibility: how to separate base UVL from BP-enhanced UVL
 - runtime integration with model synchronization

Implementation

Create GLSP for UVL & UVL-BP

Implementation

Framework Selection



Implementation

Extending UVL Metamodel

```
1 features: FEATURES_KEY NEWLINE INDENT feature envConfigFeature? envConfigFeature?
   DEDENT;
2
3 envConfigFeature:
4   reference attributes? NEWLINE;
5
6 constraint:
7   ...
8   | bpConstraint # BPEventConstraint
9   | ...;
10
11 bpConstraint:
12   requestedConstraint
13   | blockedConstraint
14   | waitedForConstraint
15   | selectedConstraint
16   | conflictingConstraint;
17
18 conflictingConstraint:
19   CONFLICTING_KEY OPEN_PAREN (reference COMMA)* reference CLOSE_PAREN;
```

Implementation

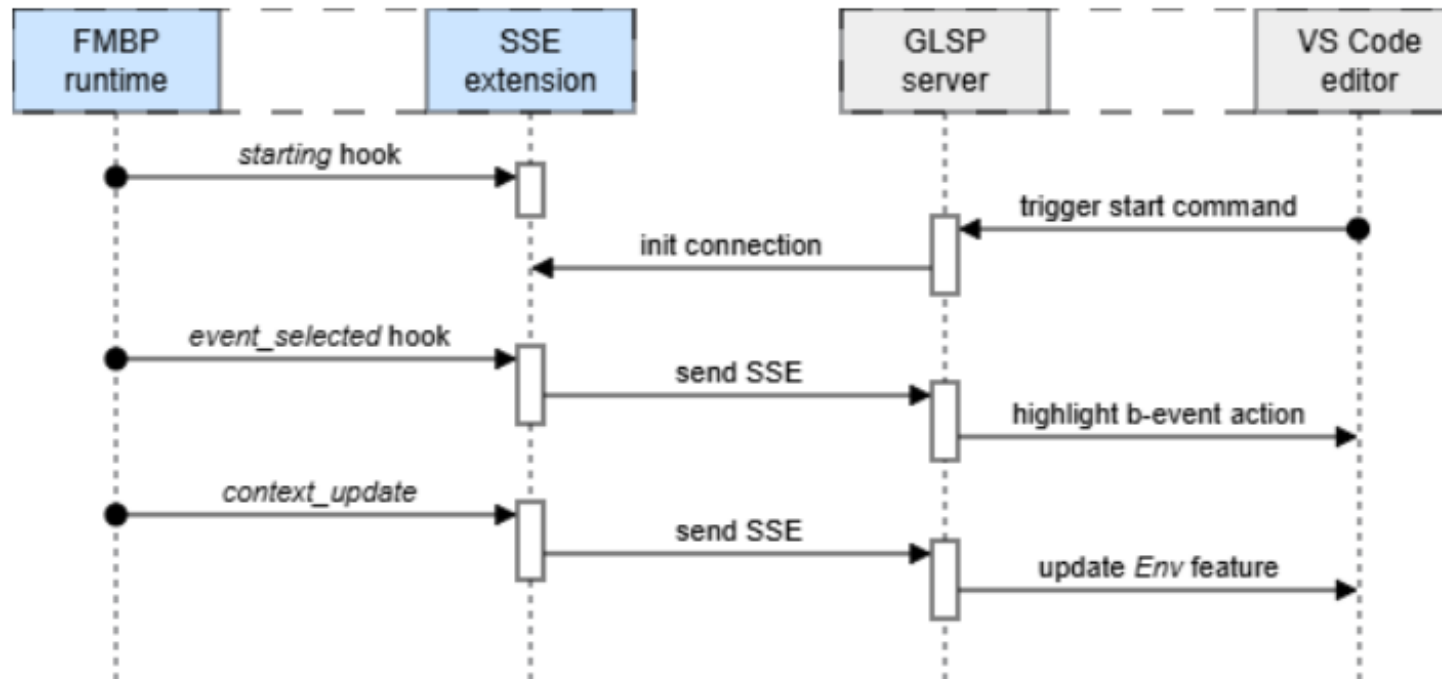
Extending UVL Metamodel

```
1 public class BPFeatureModel extends FeatureModel {
2     private Feature env;
3     private Feature config;
4
5     public Feature getEnv() { return env; }
6     public void setEnv(Feature env) { this.env = env; }
7
8     public Feature getConfig() { return config; }
9     public void setConfig(Feature config) { this.config = config; }
10
11     ...
12 }
13
14 public class ConflictingConstraint extends Constraint {
15     private final List<VariableReference> references;
16
17     public ConflictingConstraint(List<VariableReference> references) { ... }
18
19     public List<VariableReference> getReferenceList() {
20         return Collections.unmodifiableList(references);
21     }
22     public void setReference(int index, VariableReference reference) {
23         this.references.set(index, reference);
24     }
25     @Override
26     public String toString(boolean withSubmodels, String currentAlias) { ... }
27 }
```

Implementation

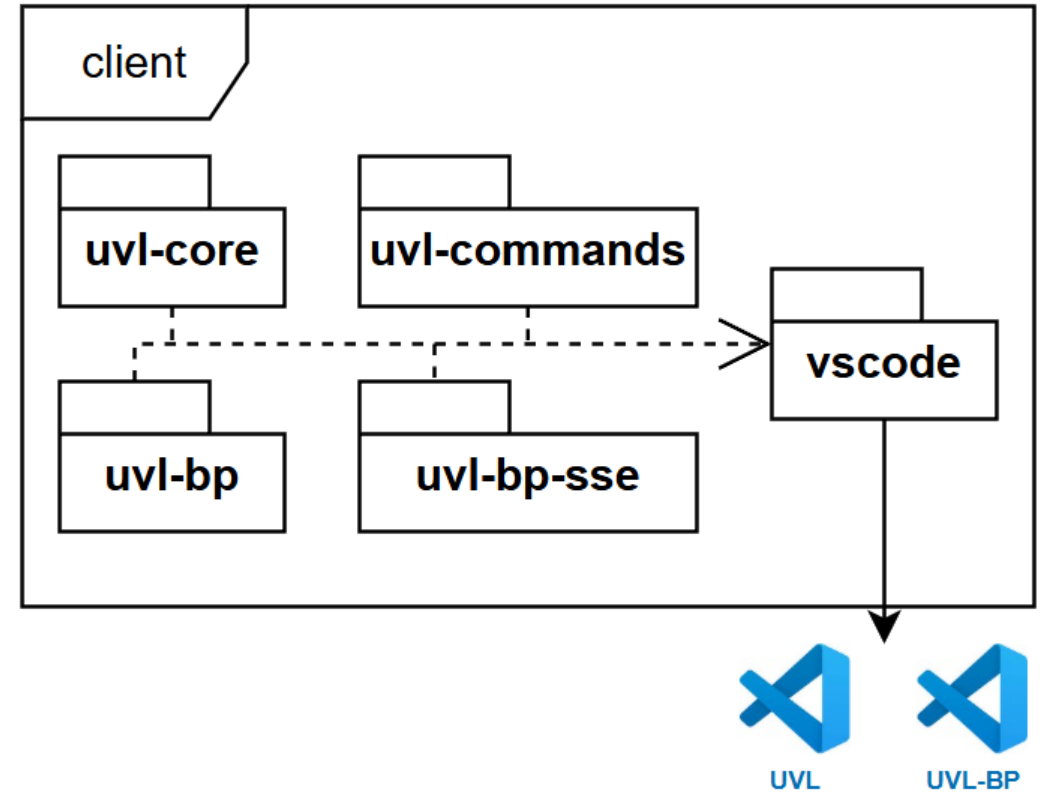
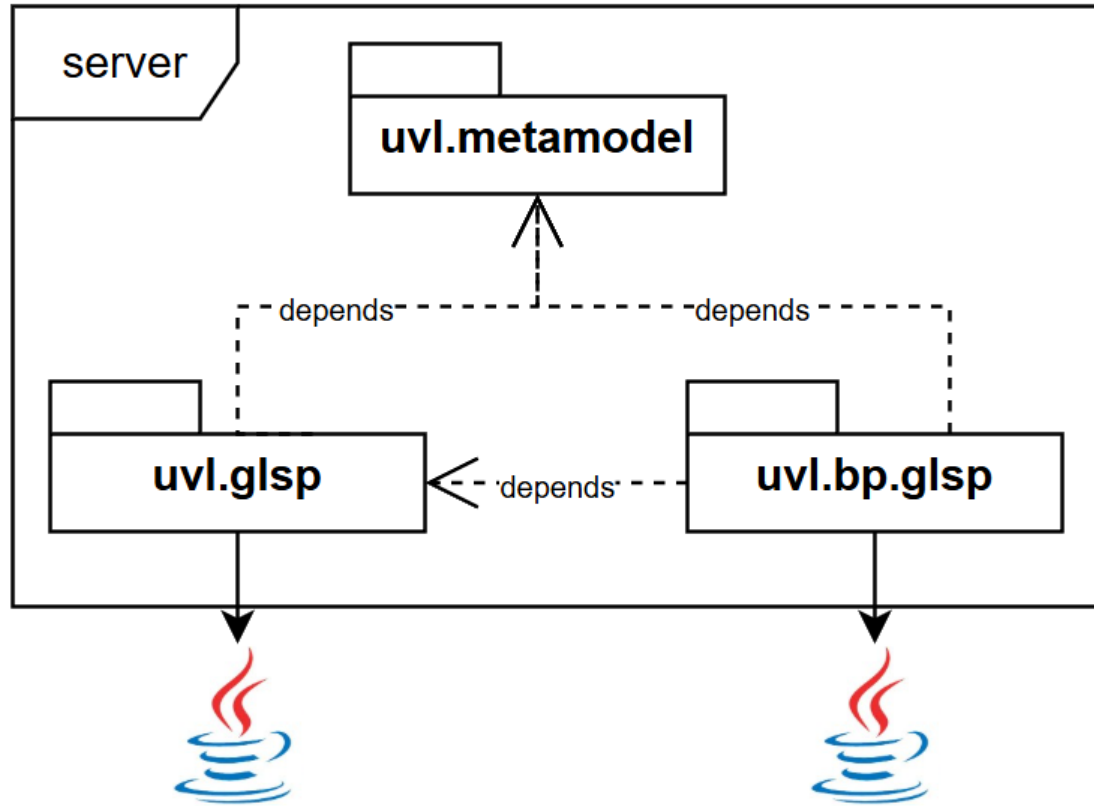
Extending FMBP

- selected b-events → extension based on *BPConfigurator*
- dynamic environment attributes → decorator for context sources
- single SSE server sending model path



Implementation

GLSP package structure



Implementation

Extending uvl.glsp to uvl.bp.glsp

```
1 public class UVLDiagramModule extends DiagramModule {
2     @Override
3     protected Class<? extends SourceModelStorage> bindSourceModelStorage() {
4         return UVLSourceModelStorage.class;
5     }
6
7     @Override
8     protected Class<? extends UVLModelState> bindGModelState() {
9         return UVLModelStateImpl.class;
10    }
11
12    @Override
13    protected void configureOperationHandlers(final MultiBinding<OperationHandler
14        <?>> binding) {
15        binding.add(UVLApplyLabelEditOperationHandler.class);
16        binding.add(UVLDeleteOperationHandler.class);
17        binding.add(UVLCreateFeatureOperationHandler.class);
18        binding.add(UVLCreateAttributeOperationHandler.class);
19        binding.add(UVLCreateRelationEdgeOperationHandler.class);
20        ...
21    }
22    ...
23 }
```

Implementation

Extending uvl.glspl to uvl.bp.glspl

```
1 public class BPDiagramModule extends UVLDiagramModule {
2     @Override
3     protected Class<? extends SourceModelStorage> bindSourceModelStorage() {
4         return BPSourceModelStorage.class;
5     }
6
7     @Override
8     protected Class<? extends BPMModelState> bindGModelState() {
9         return BPMModelStateImpl.class;
10    }
11
12    @Override
13    protected void configureOperationHandlers(final MultiBinding<OperationHandler
14        <?>> binding) {
15        super.configureOperationHandlers(binding);
16
17        binding.rebind(UVLApplyLabelEditOperationHandler.class,
18            BPAApplyLabelEditOperationHandler.class);
19        binding.rebind(UVLDeleteOperationHandler.class, BPDeleteOperationHandler.
20            class);
21        binding.add(BPCreateBThreadOperationHandler.class);
22        binding.add(BPCreateEventOperationHandler.class);
23    }
24    ...
25 }
```

Overridden source model storage
and GModel state

Reuse existing operation handlers

Rebind required operation handlers

Add new BP-related handlers

Implementation

GLSP-based multi-profile VS Code extensions

```
{
  "defaultProfile": "uvl",
  "profiles": [
    {
      "id": "uvl",
      "name": "uvl-vscode-extension",
      "displayName": "UVL VS Code Extension",
      "description": "Extension for the Universal Variability Language (UVL), using a GLSP-based diagram editor inside VSCode",
      "outFile": "uvl-vscode.vsix",
      "serverJarPath": "../../server/uvl.glsp/target/uvl.glsp-0.5.2-glsp.jar",
      "containerModuleIds": ["uvl-core"],
      "commandContributionIds": ["uvl-default"]
    }
  ],
}
```

Demo

VS Code Extensions for UVL & UVL-BP

Evaluation

Contributions & Future Work

Evaluation

Contributions

1. **Feasibility:**

Graphical editor built on GLSP, supports modeling of contextual BP models

2. **Extensibility:**

Architecture separating core UVL modeling from the BP extension using profile-based approach

3.1. **Runtime synchronization: Behavioral Program → Graphical View**

Extend FMBP providing runtime information using SSE

3.2. **Runtime synchronization: Graphical View → Behavioral Program**

Working on the same UVL model, changes are applied via FMBP integration

Future Work

Improve the proposed tool

- improve performance
- architecture and maintainability through smaller modules & BP-specific grammar and metamodel
- improvements to visuals & usability
- additional editor views
- end-to-end tests via Playwright
- new extensions based on UVL

Thank you for your attention

Questions & Discussion

Bibliography

References to the literature and images

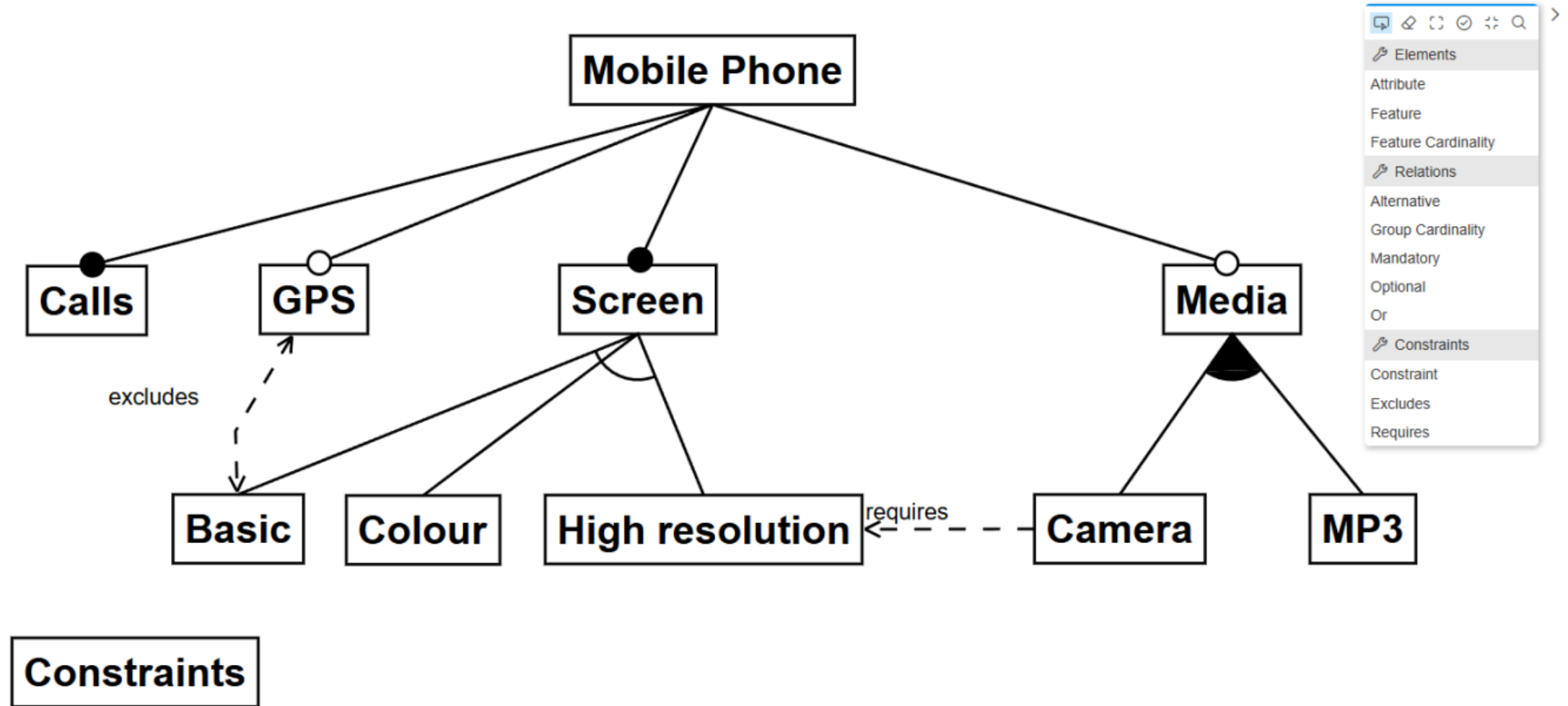
- [1] David Harel, Assaf Marron, and Gera Weiss. “Behavioral programming”. In: Commun. ACM 55.7 (July 1, 2012), pp. 90–100. ISSN: 0001-0782. DOI: 10.1145/2209249.2209270
- [2] Achiya Elyasaf. “Context-Oriented Behavioral Programming”. In: Information and Software Technology 133 (May 1, 2021). ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106504
- [3] Tom Felber and Sebastian Götz. “Behavioral Programming in the Large using Variability Management”. In: Proceedings of the 19th International Working Conference on Variability Modelling of Software-Intensive Systems. VaMoS ’25. New York, NY, USA: Association for Computing Machinery, May 28, 2025, pp. 126–130. ISBN: 979-8-4007-1441-2. DOI: 10.1145/3715340.3715443
- [4] David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. “Automated analysis of feature models 20 years later: A literature review”. In: Information Systems 35.6 (Sept. 1, 2010), pp. 615–636. ISSN: 0306-4379. DOI: 10.1016/j.is.2010.01.001
- [5] David Benavides et al. “UVL: Feature modelling with the Universal Variability Language”. In: Journal of Systems and Software 225 (July 1, 2025). ISSN: 0164-1212. DOI: 10.1016/j.jss.2024.112326
- [6] Dominik Bork, Philip Langer, and Tobias Ortmayr. “A Vision for Flexible GLSP-Based Web Modeling Tools”. In: The Practice of Enterprise Modeling. Ed. by João Paulo A. Almeida et al. Cham: Springer Nature Switzerland, 2024, pp. 109–124. ISBN: 978-3-031-48583-1. DOI: 10.1007/978-3-031-48583-1_7
- [7] FeatureIDE. <https://featureide.de/> (22.06.2026)
- [8] Nir Eitan et al. “On Visualization and Comprehension of Scenario-Based Programs”. In: 2011 IEEE 19th International Conference on Program Comprehension. 2011 IEEE 19th International Conference on Program Comprehension. ISSN: 1092-8138. June 2011, pp. 189–192. DOI: 10.1109/ICPC.2011.10
- [9] Tom Felber and Sebastian Götz. “Debugging Behavioral Programs Using Models@run.time”. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). ISSN: 2376-9521. Aug. 2024, pp. 160–167. DOI: 10.1109/SEAA64295.2024.00032
- [10] Tom Felber and Sebastian Götz. “Comprehensible Self-Aware Behavioral Programming”. In: 2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). 2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). Sept. 2025, pp. 215–220. DOI: 10.1109/ACSOS-C66519.2025.00064

Appendix

Additional slides

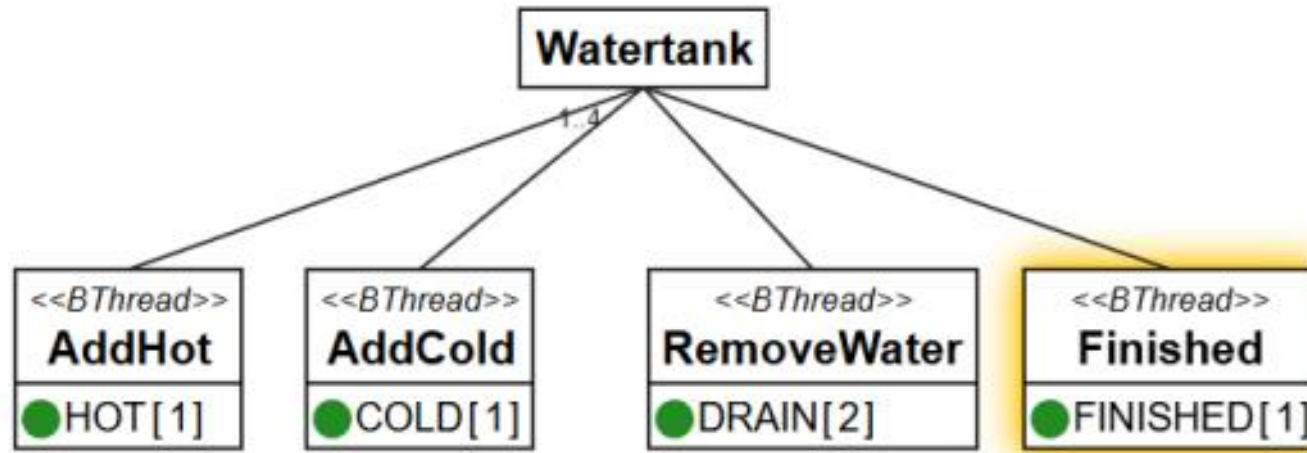
GLSP Example Model

Mobile Phone



GLSP Example Model

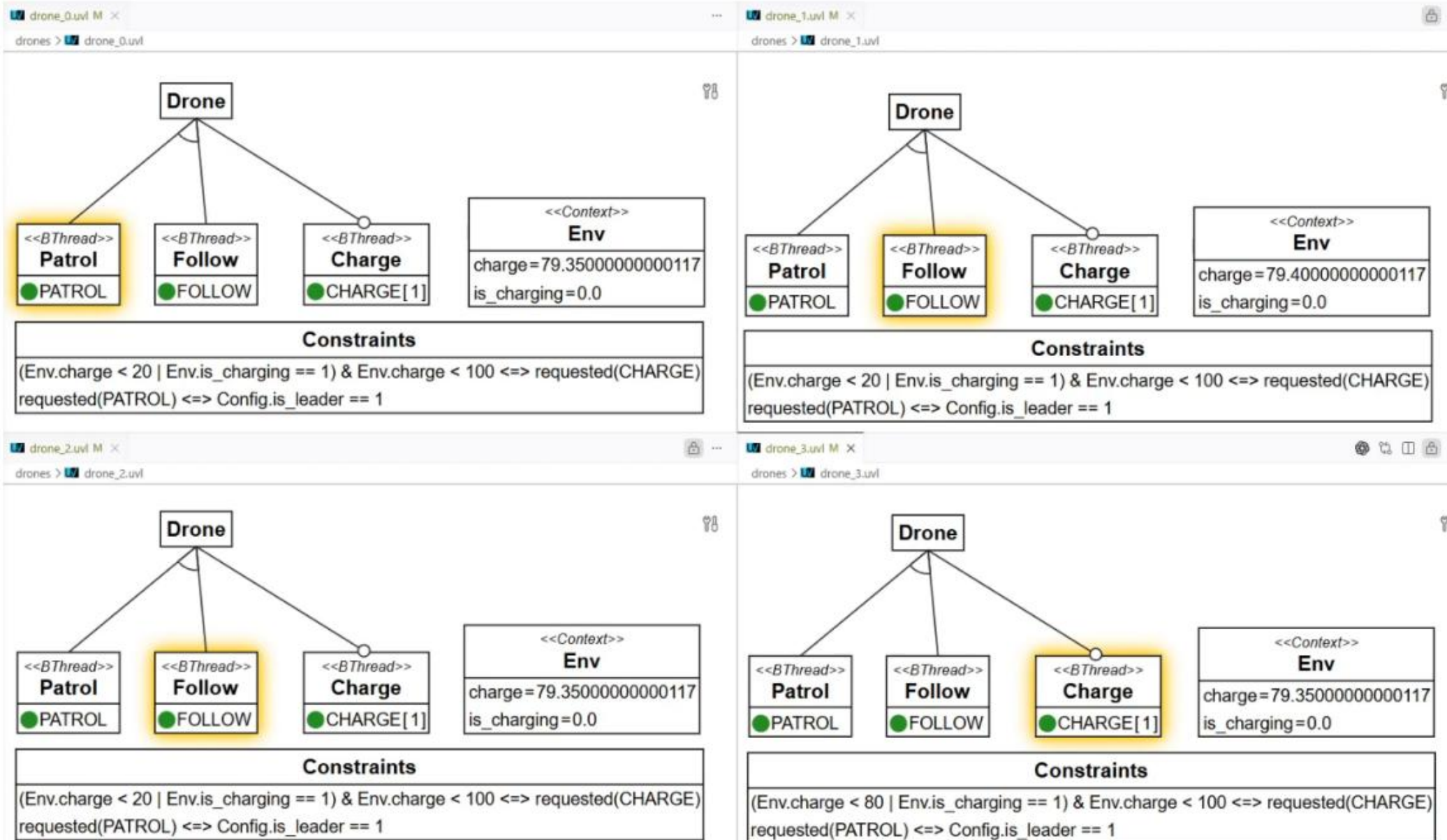
Water Tank



Constraints
conflicting(HOT, COLD, DRAIN, FINISHED) (Env.temp < Config.target_temp) => requested(HOT) (Env.temp > Config.target_temp) => requested(COLD) (Env.level > Config.target_level) <=> selected(DRAIN) (Env.temp == Config.target_temp & Env.level == Config.target_level) <=> selected(FINISHED)

<<Context>> Config	<<Context>> Env
target_temp=20	temp=20.0
target_level=50	level=50.0

GLSP Example Model Drones



Visual Representation Theory

Semiotic Clarity: Each semantic construct should map to one distinct graphical symbol; violations produce redundancy, overload, excess, or deficit.

Perceptual Discriminability: Symbols must be easy to tell apart; discriminability depends on the "visual distance" across variables such as shape, color, and size.

Semantic Transparency: Symbols should visually suggest their meaning so that appearance provides intuitive cues and reduces learning effort.

Complexity Management: Notations should offer mechanisms (for example, layered views) to present information incrementally and avoid cognitive overload.

Cognitive Integration: Provide contextual anchors, overview maps, and cross-references so readers can relate individual diagrams to the larger system of representations.

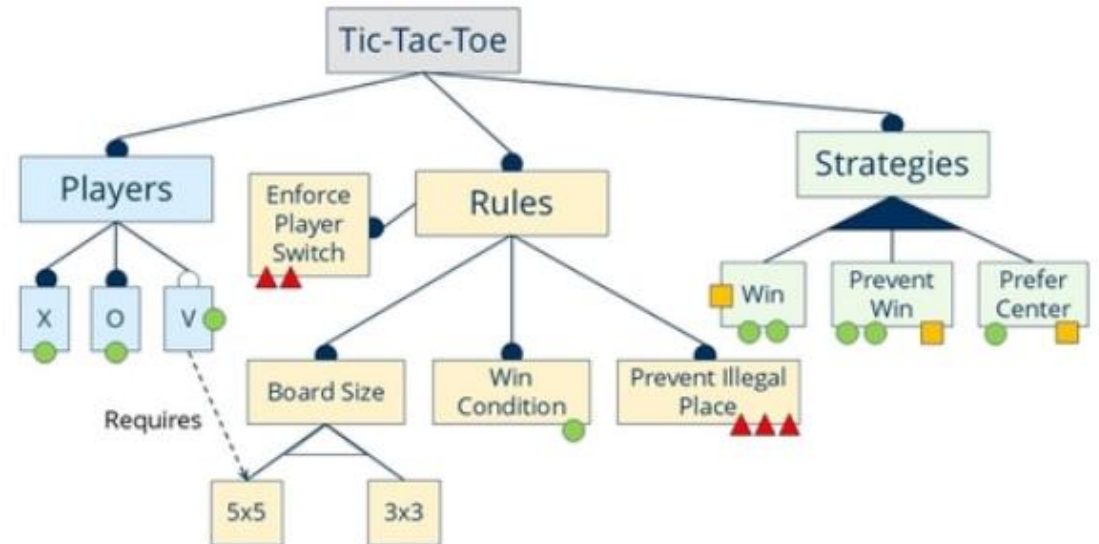
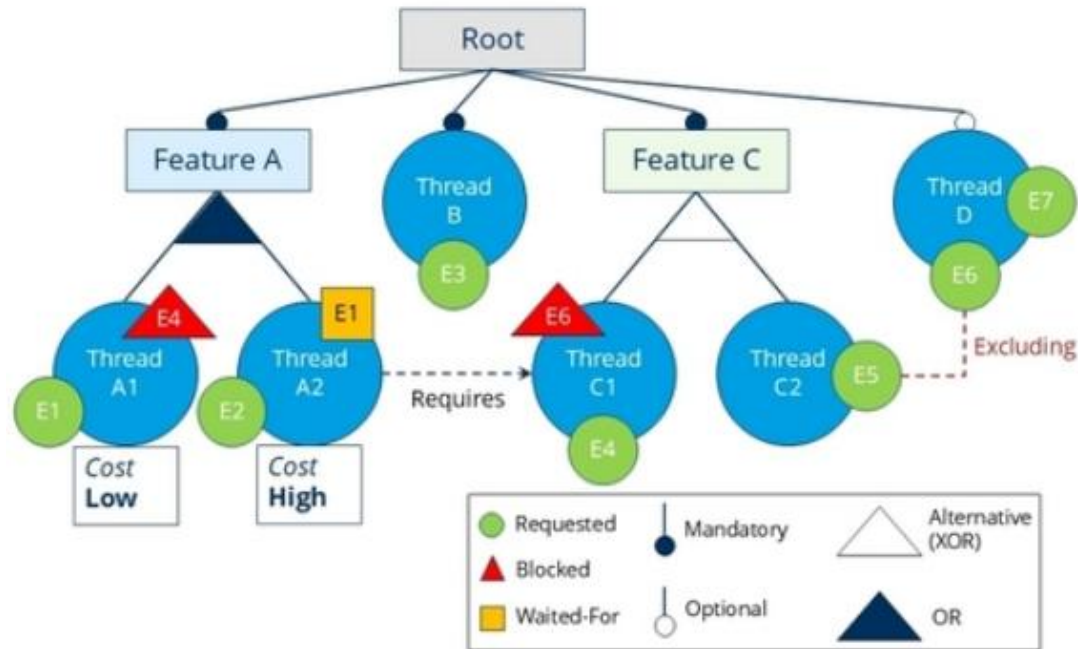
Visual Expressiveness: Leverage multiple visual channels (color, size, brightness, texture, position, etc.) so the notation can encode information across a richer perceptual space.

Dual Coding: Combine concise textual labels or annotations with graphics to leverage both verbal and visual cognitive systems and improve comprehension and retention.

Graphic Economy: Keep the visual vocabulary compact so distinct symbols remain within human discriminative abilities, especially for novice users.

Cognitive Fit: Tailor the notation to task and audience, offering simplified and expert variants so the representation matches users' goals and skills.

B-Event Visualisation Comparison



<<BThread>>

Feature

- Requested_Event[1]
- Waited-For_Event[2]
- ▲ Blocked_Event[3]

FMBP Extension

Context Source Decorator

```
1 class ContextSourceStreamDecorator(ContextSource):
2     def __init__(
3         self,
4         source: str,
5         context_source: ContextSource,
6         event_stream_server: EventStreamServer,
7     ) -> None:
8         self.__source = source
9         self.__context_source = context_source
10        self.__event_stream_server = event_stream_server
11
12    def get_data(self) -> CONTEXT_DATA:
13        data = self.__context_source.get_data()
14        self.__event_stream_server.publish({
15            "timestamp": datetime.now(timezone.utc).isoformat(),
16            "source": self.__source,
17            "type": "context_update",
18            "data": data,
19        })
20        return data
```

FMBP Extension

BEventStreamerExtension

```
1 class BEventStreamerExtension(BPConfiguratorExtension):
2     ...
3     def starting(self, b_program: BProgram) -> None:
4         self.__event_stream_server.start()
5
6     def event_selected(self, b_program: BProgram, event: BEvent) -> None:
7         ...
8         for ticket in b_program.tickets:
9             request: BEvent | None = ticket.get("request")
10            block: BEvent | None = ticket.get("block")
11            wait_for: BEvent | None = ticket.get("waitFor")
12            thread_name: str = b_program.get_name(ticket["bt"])
13
14            if request and request.name == event.name:
15                self.__publish_event("requested", thread_name, event.name)
16            if block and block.name == event.name:
17                self.__publish_event("blocked", thread_name, event.name)
18            if wait_for and wait_for.name == event.name:
19                self.__publish_event("waited_for", thread_name, event.name)
20
```

FMBP Extension

EventStreamFactory

```
1 class EventStreamFactory:
2     def __init__(self, port: int = 8099) -> None:
3         self.__server = EventStreamServer(port=port)
4
5     def b_event_stream(self, source: str) -> BEventStreamerExtension:
6         return BEventStreamerExtension(source, self.__server)
7
8     def context_source(
9         self, source: str, context_source: ContextSource
10    ) -> ContextSourceStreamDecorator:
11        return ContextSourceStreamDecorator(source, context_source, self.__server)
```

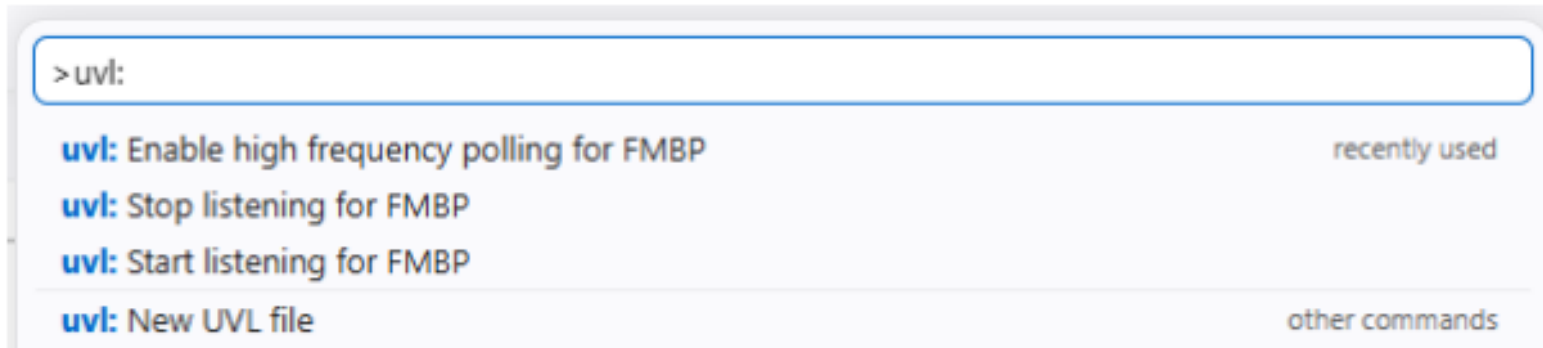
GLSP Client

UVL Diagram Module

```
1  const uvlDiagramModule = new ContainerModule((bind, unbind, isBound, rebind) => {
2    const context = {bind, unbind, isBound, rebind};
3    ...
4    bindAsService(context, TYPES.IAnchorComputer, CenteredAnchor);
5    ...
6    bind(HighlightElementsActionHandler).toSelf().inSingletonScope();
7    configureActionHandler(context, HighlightElementAction.KIND,
8      HighlightElementsActionHandler);
9
10   configureDefaultModelElements(context);
11   ...
12   configureModelElement(context, UVLModelTypes.FEATURE, FeatureNode,
13     FeatureNodeView);
14   configureModelElement(context, UVLModelTypes.ATTRIBUTE, EditableGCompartment,
15     GCompartmentView);
16   configureModelElement(context, UVLModelTypes.REQUIRES, GEdge,
17     SingleArrowEdgeView);
18   ...
19 });
```

Models@run.time in GLSP

Commands & Injection in GLSP Server



```
1 @Override
2 protected void configureAdditional() {
3     super.configureAdditional();
4
5     bind(ServerSentEventsService.class).to(FMBPEventListenerService.class).
        asEagerSingleton();
6     bind(FMBPHighlightActionDispatchService.class).asEagerSingleton();
7     bind(FMBPContextUpdateService.class).asEagerSingleton();
8 }
```

Models@run.time in GLSP

Highlight dispatch

```
1 @Inject
2 protected void registerDataListener() {
3     serverSentEventsService.addDataListener(this::dispatchHighlightAction, "
4         requested", "blocked", "waited_for");
5 }
6
7 protected void dispatchHighlightAction(final ParsedServerSentEvent event) {
8     Optional<String> threadName = parseThreadName(event);
9     Optional<String> eventName = parseEventName(event);
10
11     Set<String> matchingFeatureIds = eventName.isPresent()
12         ? resolveFeatureIdsByEventName(threadName.get(), eventName.get())
13         : resolveFeatureIdsByThreadName(threadName.get());
14
15     actionDispatcher.dispatch(
16         new HighlightElementAction(List.copyOf(matchingFeatureIds), true));
17 }
```

Models@run.time in GLSP

Context Update

```
1 @Inject
2 protected void registerDataListener() {
3     serverSentEventsService.addDataListener(this::updateContextEnv,
4         CONTEXT_UPDATE_TYPE);
5 }
6
7 protected void updateContextEnv(final ParsedServerSentEvent event) {
8     Feature envFeature = modelState.getFeatureModel().getEnv();
9     ...
10    for (Map.Entry<String, ?> entry : event.data().entrySet()) {
11        if (envFeature.getAttributes().containsKey(entry.getKey())) {
12            Attribute attribute = envFeature.getAttributes().get(entry.getKey());
13            if (!Objects.equals(attribute.getValue(), entry.getValue())) {
14                attribute.setValue(entry.getValue());
15                hasChanges = true;
16            }
17        }
18    }
19    if (hasChanges) actionDispatcher.dispatchAll(submitModel(envFeature));
20 }
```